

Task 1: Understand React and Its Advantages

- a) Write a brief explanation of React's core features: component-based architecture, Virtual DOM, and unidirectional data flow.
- b) Discuss why these features make React ideal for building modern web applications.

Component-based architecture

Component-based architecture is an approach where UI is broken down into reusable components. It allows users to create encapsulated components that manage their own state.

- These components can be functional, or class based.
- Each component manages its own logic and rendering and thus Component-based architecture promotes reusability, maintainability, and scalability.

Virtual DOM

The Virtual DOM is a lightweight in-memory representation of the real DOM. When a component's state changes, React updates the Virtual DOM first, calculates the difference, and then efficiently updates only the necessary parts of the real DOM.

Virtual DOM improves performance by minimizing direct manipulation of the actual DOM. The number of operations on actual DOM are reduced. Thus, it makes the UI more responsive.

- Creates a lightweight copy of the DOM (Virtual DOM).
- Compares it with the previous version to detect changes (diffing).
- Updates only the changed parts in the actual DOM (reconciliation), improving performance.

Unidirectional data flow

React uses one way data binding, meaning data flows in one direction—from parent components to child components via props. This one-way binding simplifies data management, making the flow of data predictable and easier to debug.

These features make React ideal for building modern web applications:

Scalability & Maintainability: The component-based approach allows developers to build scalable applications by reusing and composing components. It makes code easier to maintain and test.

Performance Optimization: The Virtual DOM minimizes direct interactions with the real DOM, leading to faster rendering and better performance. This approach is crucial for dynamic and interactive user interfaces.

Predictability & Debugging: Unidirectional data flow provides a clear path for how data moves through the app, making it easier to track state changes and debug issues. This is particularly beneficial in complex applications with many interdependent components.

Task 2: Set Up a Basic React Environment

- Create an HTML file and include CDN links for React and ReactDOM.
- Add a `<div>` with an id of "root" in the HTML body.
- Write a simple React component inside a `<script>` tag and use `ReactDOM.render()` to render it to the page.

```
Task2.html U X
Module 8 > Module 8_Week 9_Required Assignment_Patil > Task2 > Task2.html > ...
1  <!DOCTYPE html>
2  <html lang="en">
3  <head>
4      <meta charset="UTF-8">
5      <meta name="viewport" content="width=device-width, initial-scale=1.0">
6      <title>NUS Module 8 - Basic React</title>
7
8      <!-- React and ReactDOM CDN -->
9      <script crossorigin src="https://unpkg.com/react@18/umd/react.development.js"></script>
10     <script crossorigin src="https://unpkg.com/react-dom@18/umd/react-dom.development.js"></script>
11
12     <!-- Babel for JSX transformation -->
13     <script src="https://unpkg.com/@babel/standalone/babel.min.js"></script>
14 </head>
15 <body>
16
17     <!-- Create a div with id of root-->
18     <div id="root">
19
20     </div>
21
22     <!-- Create a react componet inside script tag-->
23     <script type="text/babel">
24         function App(){
25             return(
26                 <>
27                     <h1> Welcome to NUS Module 8 </h1>
28                     <h3> Task 2 : Set Up a Basic React Environment </h3>
29                     <p> This is a basic setup for react. This is an html page which renders some text using basic html tags.</p>
30                 </>
31             )
32         }
33
34         const root = ReactDOM.createRoot(document.querySelector("#root"));
35         root.render(
36             <App/>
37         )
38     </script>
39 </body>
40 </html>
```

The above screenshot shows code in an html file which contains the links for React and ReactDOM CDN, as well as the link for babel for JSX transformation. There is a functional component in the script tag, which returns a JSX element. Then React.createRoot is used to render this App element inside 'root'.

I have used 'createRoot' instead of ReactDOM.render() as the 'createRoot' is introduced from React 18 onwards.

The output of the code is as below.



Task 3: Create and Render Functional Components

- a) Define a functional React component (e.g., GreetingComponent) that returns a simple JSX element.
- b) Render the component using ReactDOM.render() to display it on the page.



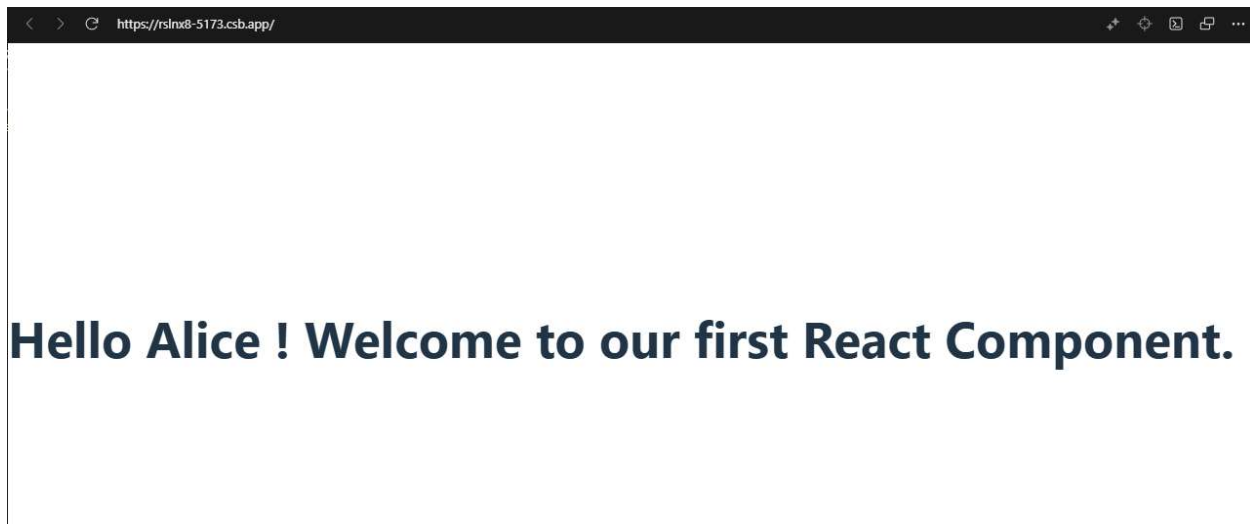
The above screenshot shows the 'GreetingComponent' which accepts 'name' as props and returns the greeting message using JSX.

```
main.jsx M X
src > main.jsx
1 import { StrictMode } from "react";
2 import { createRoot } from "react-dom/client";
3 import GreetingComponent from "../Components/GreetingComponent";
4 import "../index.css";
5
6 createRoot(document.getElementById("root")).render(
7   [
8     <StrictMode>
9       <GreetingComponent name="Alice" />
10    </StrictMode>
11  );
```

The above screenshot shows the main.jsx which renders the 'GreetingComponent' using 'createRoot'.

I have used 'createRoot' instead of ReactDOM.render() as the 'createRoot' is introduced from React 18 onwards.

The output is as below:



Task 4: Use JSX for Structuring Components

- Create a component that renders a list of items (using an unordered list) and a heading (e.g., "My To-Do List").
- The component should display at least three items in the list using JSX.

```
ToDoList.jsx M X
src > Components > ToDoList.jsx > ...
1  import React from "react";
2
3  function ToDoList() {
4    return (
5      <>
6        <h1>My To-Do List</h1>
7        <ul>
8          <li>Task 1 : Learn hot to Setup a new project using React</li>
9          <li>Task 2 : Learn JSX basics</li>
10         <li>Task 3 : Learn using props in React </li>
11         <li>Task 4 : Learn hooks in React</li>
12         <li>Task 5 : Build a sample application</li>
13       </ul>
14     </>
15   );
16 }
17
18 export default ToDoList;
19
```

The above screenshot shows a ToDoList component that renders a heading and the list of a few unordered list items.

```
main.jsx M X
src > main.jsx
1  import { StrictMode } from "react";
2  import { createRoot } from "react-dom/client";
3  import GreetingComponent from "../Components/GreetingComponent";
4  import ToDoList from "../Components/ToDoList";
5  import "../index.css";
6
7  createRoot(document.getElementById("root")).render(
8    <StrictMode>
9      <GreetingComponent name="Alice" />
10     <ToDoList />
11   </StrictMode>
12 );
13
```

The above screenshot shows the main.jsx which renders the 'ToDoList' using 'createRoot'.

The output is as below:

Hello Alice ! Welcome to our first React Component.

My To-Do List

- Task 1 : Learn how to Setup a new project using React
- Task 2 : Learn JSX basics
- Task 3 : Learn using props in React
- Task 4 : Learn hooks in React
- Task 5 : Build a sample application